

# 딥러닝 프레임워크 3종 완전 비교

Keras · PyTorch · TensorFlow

구조적 관계 · 정확도의 진실 · 알고리즘 활용 · 학습 곡선 · 환경/생태계  
동일 과제(MNIST) 3종 코드 비교 실습 포함

교육·웹 콘텐츠 제작용 정리 문서 (Claude Code 웹 페이지 제작 소스)  
2026. 07 · DreamIT Biz

# 목차

---

1. 핵심 요약 (Executive Summary)
2. 구조적 관계 — 비교하면 안 되는 것
3. 정확도의 진실 — 프레임워크는 정확도를 바꾸지 않는다
4. 알고리즘 활용 비교 — 어디에 무엇이 유리한가
5. 사용하며 배우는 차이 — 학습 곡선
6. 환경·생태계의 차이
7. 동일 과제 3종 코드 비교 실습 (MNIST 784→128→10)
8. 종합 비교표 및 실무 분기점
9. 선택 가이드 & 권장 교육 시퀀스
10. [부록] Claude Code 웹 페이지 제작 가이드

## 1. 핵심 요약 (Executive Summary)

- Keras는 프레임워크가 아니라 **고수준 API**다. "Keras vs TensorFlow"는 층위가 잘못된 비교이며, Keras 3부터는 TensorFlow·PyTorch·JAX 어느 백엔드 위에서도 동작한다.
- 올바른 비교 구도는 두 가지다: **TF vs PyTorch (엔진 대결)**, **고수준(Keras)을 쓸 것인가, 저수준 API를 직접 쓸 것인가 (추상화 수준 선택)**.
- **모델 정확도는 프레임워크 때문에 달라지지 않는다**. 동일 모델·데이터·하이퍼파라미터면 세 방식 모두 사실상 동일한 성능이 나온다. 달라지는 것은 결과가 아니라 **과정의 가시성과 개발 경험**이다.
- 실무 분기: 표준 문제·입문 교육은 Keras, 연구·최신 모델·커스텀 로직·취업 대비는 PyTorch, 모바일/웹/엣지 배포·구글 생태계는 TensorFlow.
- MNIST 동일 과제 실습 결과: Keras 약 15줄 / PyTorch 약 25줄 / TF-GradientTape 약 25줄 — **세 방식 모두 최종 정확도 약 97.7%로 동일** (시드에 따른  $\pm 0.1 \sim 0.2\%$  오차뿐).

## 2. 구조적 관계 — 비교하면 안 되는 것

세 도구는 같은 층위의 경쟁자가 아니다. 이 구조를 이해하면 나머지 비교가 전부 정리된다.

| 도구         | 층위      | 개발 주체   | 정체성                                                                |
|------------|---------|---------|--------------------------------------------------------------------|
| TensorFlow | 저수준 엔진  | 구글      | 텐서 연산·자동 미분·분산 처리·배포 인프라를 포함한 생태계 전체                               |
| PyTorch    | 저수준 엔진  | 메타      | TF와 동일 층위의 경쟁자. Define-by-run(동적 그래프)의 원조                          |
| Keras      | 고수준 API | (멀티백엔드) | TF 2.x에서 tf.keras로 흡수 → Keras 3(2023~)부터 TF·PyTorch·JAX 멀티백엔드로 재독립 |

※ 강의에서 반드시 언급할 최신 변화: Keras 3부터 PyTorch가 Keras의 "경쟁자"가 아니라 "백엔드"가 될 수 있는 관계로 바뀌었다.

### 올바른 비교 구도

- 질문 1 — **엔진 대결**: TensorFlow vs PyTorch (동일 층위 경쟁)
- 질문 2 — **추상화 선택**: Keras(고수준)를 쓸 것인가, 저수준 API를 직접 쓸 것인가

## 3. 정확도의 진실 — 프레임워크는 정확도를 바꾸지 않는다

"파이토치가 정확도가 더 높다" 같은 말은 잘못된 통념이다. 세 도구 모두 동일한 수학(역전파, 동일 옵티마이저, 동일 아키텍처)을 구현하므로, 같은 모델·같은 데이터·같은 하이퍼파라미터면 이론상 동일한 성능이 나온다.

### 실무에서 미세한 차이가 나는 실제 원인 4가지

- 가중치 초기화 기본값의 차이 (레이어별 기본 initializer가 프레임워크마다 다름)
- 옵티마이저 기본 하이퍼파라미터 차이 (epsilon 값 등)
- 데이터 증강·전처리 파이프라인 구현 차이
- 랜덤 시드와 비결정적(non-deterministic) GPU 연산

## 단, "성능 접근성"은 다르다

연구 논문의 최신 모델(SOTA) 구현체는 PyTorch가 압도적으로 많다. "정확도 자체"가 아니라 "**최신 모델을 빨리 가져다 쓸 수 있는가**"라는 실용적 접근성에서는 PyTorch가 유리하다.

## 4. 알고리즘 활용 비교 — 어디에 무엇이 유리한가

| 활용 영역                       | 유리한 쪽             | 이유                                           |
|-----------------------------|-------------------|----------------------------------------------|
| 최신 연구 모델 (LLM, Diffusion 등) | <b>PyTorch</b>    | 논문 구현체 대다수가 PyTorch, Hugging Face 생태계 중심     |
| 커스텀 아키텍처·동적 구조              | <b>PyTorch</b>    | Define-by-run 철학의 원조 — 가변 길이·재귀 구조 제어가 자연스러움 |
| 표준 CNN/RNN/분류·회귀            | <b>Keras</b>      | 몇 줄로 완성, 오류 여지 최소                            |
| 모바일·엣지 배포                   | <b>TensorFlow</b> | TFLite(현 LiteRT)가 성숙, 안드로이드 통합 우수            |
| 웹 브라우저 배포                   | <b>TensorFlow</b> | TensorFlow.js — 웹 개발과의 직접 연결점                |
| 대규모 프로덕션 서빙                 | <b>양쪽 성숙</b>      | TF Serving vs TorchServe/ONNX — 현재 격차 거의 없음  |
| TPU 활용                      | <b>TF / JAX</b>   | 구글 TPU 지원이 우선됨                               |

## 5. 사용하며 배우는 차이 — 학습 곡선

### Keras — 개념 학습에 최적

Sequential → compile() → fit() 3단계로 끝난다. 학습자가 "딥러닝의 개념 흐름"에 집중할 수 있어 입문 교육에 최적이다. 대신 내부가 블랙박스라 역전파·학습 루프의 실제 동작을 체감하기 어렵다.

### PyTorch — 원리 학습에 최적

학습 루프를 직접 작성한다: zero\_grad() → forward → loss.backward() → optimizer.step(). 코드가 길어지지만 "학습이 실제로 어떻게 일어나는지"가 코드에 그대로 드러난다. 파이썬다운(Pythonic) 코드라서 print()와 중단점만으로 디버깅이 직관적이다.

### TensorFlow 저수준 API — 함정 주의

TF 1.x 시절의 세션·정적 그래프 악명은 TF 2.x의 eager execution으로 해소됐다. 그러나 API가 여러 세대에 걸쳐 누적돼 있어, 검색 시 구버전 코드가 섞여 나오는 것이 학습자에게 가장 큰 함정이다.

### 교육 설계 관점

- 자연스러운 시퀀스: Keras로 개념 → PyTorch로 원리
- 실무형 과정(예: SKALA)에서는 처음부터 PyTorch로 가는 것도 현재 업계 표준에 부합

## 6. 환경·생태계의 차이

| 항목           | PyTorch                         | TensorFlow                                     |
|--------------|---------------------------------|------------------------------------------------|
| 설치·CUDA 매칭   | 공식 설치 셀렉터로 더 편리                 | CUDA 버전 매칭 번거로움                                |
| 윈도우 네이티브 GPU | 지원 (사고 적음)                      | TF 2.10 이후 중단 — WSL2 필요                        |
| 애플 실리콘       | MPS 백엔드 (더 매끄러움)                | tensorflow-metal 플러그인                          |
| 디버깅          | 일반 파이썬 디버깅 그대로                  | @tf.function 그래프화 시 난도 상승                      |
| 시각화          | TensorBoard 그대로 사용 가능           | TensorBoard 원조 (실무는 W&B; MLflow로 수렴 → 사실상 무승부) |
| 커뮤니티 추세      | 연구·스타트업·오픈소스 쏠림 뚜렷, 채용 요구 비중 증가 | 구글 생태계·모바일·기존 기업 프로덕션에서 강세                     |

※ 윈도우 실습 환경으로 강의를 구성한다면 PyTorch가 환경 사고가 적다. 맥북(애플 실리콘) 환경이라면 PyTorch MPS가 좀 더 매끄럽다.

## 7. 동일 과제 3종 코드 비교 실습

**과제:** MNIST 손글씨 분류 — 동일 모델 구조  $784 \rightarrow 128(\text{ReLU}) \rightarrow 10$ . 같은 문제를 푸는 코드가 어떻게 다른지 나란히 보면 차이가 바로 체감된다. (실행 가능한 전체 코드는 첨부 .py 파일 3종 참조)

### Keras — "무엇을 만들지"만 선언 (약 15줄)

```
import keras
from keras import layers

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = keras.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, batch_size=32)
model.evaluate(x_test, y_test)
```

학습 루프가 아예 보이지 않는다. fit() 안에서 순전파·역전파·가중치 갱신이 전부 자동으로 일어난다.

### PyTorch — "어떻게 학습하는지"를 직접 작성 (약 25줄)

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

transform = transforms.ToTensor()
train_ds = datasets.MNIST('.', train=True, download=True, transform=transform)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 10)      # softmax 없음 — 손실함수에 포함됨
)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

for epoch in range(5):
    for x, y in train_loader:
        optimizer.zero_grad()      # ① 기울기 초기화
        pred = model(x)            # ② 순전파
        loss = criterion(pred, y)   # ③ 손실 계산
        loss.backward()            # ④ 역전파
        optimizer.step()           # ⑤ 가중치 갱신
```

Keras의 fit() 한 줄이 하단 5단계 루프로 풀어져 있다. **이 5줄이 딥러닝 학습의 실체**이고, PyTorch는 이것을 숨기지 않는다. 디버깅할 때 루프 안에 print(loss.item())을 그냥 넣으면 된다.

## TensorFlow 저수준 (GradientTape) — 미분 과정까지 노출 (약 25줄)

```
import tensorflow as tf

(x_train, y_train), _ = tf.keras.datasets.mnist.load_data()
x_train = (x_train / 255.0).astype('float32')

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
optimizer = tf.keras.optimizers.Adam()

dataset = tf.data.Dataset.from_tensor_slices(
    (x_train, y_train)).shuffle(60000).batch(32)

for epoch in range(5):
    for x, y in dataset:
        with tf.GradientTape() as tape:          # 연산 기록 시작
            pred = model(x, training=True)
            loss = loss_fn(y, pred)
            grads = tape.gradient(loss, model.trainable_variables) # 미분
            optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

PyTorch와 구조는 같지만, 기울기를 `tape.gradient()`로 **명시적으로 꺼내서** 옵티마이저에 넘긴다. PyTorch는 `loss.backward()`가 기울기를 텐서에 자동으로 심어주는 반면, TF는 "기록 → 미분 → 적용"이 분리돼 있어 미분 과정 자체를 가르치기엔 오히려 더 노골적이다.

## 8. 종합 비교표 및 실무 분기점

| 비교 항목            | Keras                       | PyTorch                      | TF 저수준                        |
|------------------|-----------------------------|------------------------------|-------------------------------|
| 코드 분량            | 약 15줄                       | 약 25줄                        | 약 25줄                         |
| 학습 루프            | 숨김 (fit)                    | 직접 작성                        | 직접 작성                         |
| 역전파 표현           | 안 보임                        | <code>loss.backward()</code> | <code>tape.gradient()</code>  |
| softmax 위치       | 모델 마지막 층                    | 손실함수에 내장                     | <code>from_logits=True</code> |
| 커스터마이징           | 콜백/서브클래싱 필요                 | 루프에 코드 추가                    | 루프에 코드 추가                     |
| 에러 추적            | <code>fit()</code> 내부 (어려움) | 해당 줄 (쉬움)                    | tape 블록 (중간)                  |
| 최종 정확도 (5 epoch) | 약 97.7%                     | 약 97.7%                      | 약 97.7%                       |

※ 마지막 행이 핵심의 증명: 세 코드 모두 수학적으로 동일한 모델이라 정확도는 사실상 같다 (시드에 따른  $\pm 0.1 \sim 0.2\%$  오차분). 달라지는 것은 결과가 아니라 과정의 가시성이다.

### 세 코드가 진짜 갈리는 지점 — 비표준 요구사항

예: "배치마다 손실이 특정 값을 넘으면 그 샘플만 따로 저장하라"는 요구가 온다면:

- **Keras:** 커스텀 콜백 클래스를 만들거나 `train_step`을 오버라이드해야 함 — Keras 내부 구조 지식 필요
- **PyTorch / TF 저수준:** 루프 안에 `if loss > threshold:` 한 줄 추가하면 끝

결론: 표준 문제는 Keras가 빠르고, 비표준 문제는 저수준이 빠르다. 연구·최신 모델 작업이 PyTorch로 쏠린 것도 결국 이 유연성 때문이다.

## 9. 선택 가이드 & 권장 교육 시퀀스

| 상황                       | 권장 선택                    |
|--------------------------|--------------------------|
| 입문 교육, 빠른 프로토타입, 표준 문제   | Keras (백엔드는 무엇이든)        |
| 연구, 최신 모델, 커스텀 로직, 취업 대비 | PyTorch                  |
| 모바일/웹/엣지 배포, 구글 클라우드·TPU | TensorFlow               |
| 정확도가 걱정될 때               | 프레임워크가 아니라 데이터·모델·튜닝이 결정 |

### 권장 교육 시퀀스 (실전형 교수법)

- **기본 시퀀스:** Keras(개념) → PyTorch(원리·실무 표준)
- **심화 시퀀스:** Keras(개념) → GradientTape("미분이 이렇게 일어난다" 시연) → PyTorch(실무 안착)
- **실습 설계:** 동일 데이터셋(MNIST/Fashion-MNIST)으로 3방식 나란히 구현 → 코드 줄 수·학습 루프 구조·결과 정확도 비교 → "정확도는 같고 개발 경험이 다르다"를 체감으로 전달



## 10. [부록] Claude Code 웹 페이지 제작 가이드

이 문서를 소스로 웹 페이지를 제작할 때 권장하는 정보 구조(IA)와 구현 지침이다.

### 권장 페이지 구조 (단일 페이지 / 섹션 앵커 내비게이션)

| 섹션      | 콘텐츠 소스           | 권장 UI 컴포넌트                            |
|---------|------------------|---------------------------------------|
| Hero    | 문서 표지 + 1장 핵심 요약 | 타이틀 + 핵심 메시지 3줄 + 앵커 버튼               |
| 층위 구조   | 2장               | 3단 계층 다이어그램 (엔진 2개 위에 Keras)          |
| 정확도의 진실 | 3장               | 콜아웃 카드 + 원인 4가지 아이콘 리스트               |
| 활용 영역   | 4장 표             | 필터/배지형 비교 카드 그리드                      |
| 학습 곡선   | 5장               | 3열 카드 (Keras/PyTorch/TF) + 교육 시퀀스 화살표 |
| 환경 비교   | 6장 표             | 2열 대결(versus) 테이블                     |
| 코드 비교   | 7장 코드 3종         | 탭 전환형 코드 뷰어 (구문 강조, 복사 버튼)            |
| 종합 비교표  | 8장 표             | 반응형 테이블 — "정확도 동일" 행 하이라이트            |
| 선택 가이드  | 9장               | 의사결정 플로차트 또는 상황별 카드                   |

### 구현 지침

- **디자인 토큰:** primary #1F3864 (navy), secondary #2E5395, accent #C55A11, 배경 #F5F6F8 / 코드 배경 #F0F2F6 — 본 문서와 동일한 하우스 스타일 유지
- **타이포그래피:** 한글 본문 Pretendard 또는 Noto Sans KR, 코드 D2Coding 또는 JetBrains Mono (웹폰트 CDN)
- **코드 뷰어:** 3종 코드를 탭으로 전환, 학습 루프 5단계( ~ ) 주석 줄에 하이라이트 처리, 클립보드 복사 버튼 포함
- **핵심 강조 포인트:** "정확도 97.7% 동일"을 시각적 앵커로 사용 — 세 탭 어디서든 결과가 같다는 것이 페이지의 중심 메시지
- **반응형:** 모바일에서 비교표는 카드 스택으로 전환, 코드 블록은 가로 스크롤 허용
- **파일 연결:** 01\_keras\_mnist.py / 02\_pytorch\_mnist.py / 03\_tensorflow\_gradienttape\_mnist.py 3종 다운로드 버튼 제공
- **기술 스택 권장:** 정적 단일 HTML(+Tailwind) 또는 React 컴포넌트 — 강의 자료 배포용이므로 빌드 없는 단일 파일이 관리에 유리